



King AbdulAziz University

Faculty of Computing & Information Technology

Computer Science Department

Final Lab Exam (4-5-2025)

Course: CPCS-214 Computer Organization and Architecture

Term: 2025 Second Semester

Time: 90 Minutes (11am - 1pm)

15 Marks

Instructions:

- Student must complete their task and upload the solution **within 90 mins.**
- **Submission Link will close after 90 minutes**
 - Renamed template with your information: Section-ID-FinalExam, Example: (BCR-123456-FinalExam).
- Upload ONLY **.S** file on Blackboard.
- Instructor will guide you to the Exam folder path on BB.
- **Make sure to add your name / ID / Section / course name / Lab Final Exam, as comments at the beginning of your program.** It is important and **counts** in your marks.
- Mobile Phones should be switched off during the exam.
- Browsing Internet during the exam is not allowed. Only Access to BB is allowed for uploading the Exam Solution at the ending of exam.
- During exam keep saving your work.
- In case of any issue, contact the instructor immediately.

Write a complete MIPS assembly program that:

1. Data Declaration:

- Declare an array of 5 integers (**uninitialized**, space reserved).
- Declare a string prompt "Enter score: " to ask for each student score.
- Declare two strings; to hold the performance of student ("Passed" or "Failed").
- Declare an integer **comment** to hold the performance flag (0 = Fail, 1 = Pass)
- Reserve space for storing the **average score** as an integer.
- Declare any strings for messages to match the output.

○

2. Main Program Logic:

- In a **loop**:
 - Print the prompt "Enter score: " for each student.
 - Read an integer from the user and store it into the array.
- **Call a subroutine** "calculate_average" to:
 - Loop through the array.
 - Sum all the scores.
 - Calculate the average (integer division).
 - Store the average in the reserved space.
- **Call another subroutine** "evaluate_performance" to:
 - If the average score is ≥ 60 , set the comment to **1**.
 - Else, set the comment to **0**.
- **Print** the following outputs:
 - The average score.
 - The performance "Passed/Failed" based on the value of **comment**.

3. Program Structure Requirements:

- Use **proper .data and .text sections**.
- Use **system calls** for all input/output (Shown in Figure.1 below).
- Use **loops** for both **reading scores** and **summing scores**.
- Use **subroutines** (jal + jr \$ra) for "calculate_average" and "evaluate_performance".
- Correct use of **integer arithmetic** and **comparisons**.
- Code must be **clearly commented** and **well-organized**.

Sample outputs

```
Enter score: 77
Enter score: 66
Enter score: 49
Enter score: 88
Enter score: 79

The average of your scores is
71
You have Passed
-- program is finished running -
```

```
Enter score: 69
Enter score: 59
Enter score: 47
Enter score: 60
Enter score: 57

The average of your scores is
58
You have Failed
-- program is finished running -
```

System Services

Service	Code in \$vo	Arguments	Results
print_int	1	\$a0 = integer to be printed	
print_float	2	\$f12 = float to be printed	
print_double	3	\$f12 = double to be printed	
print_string	4	\$a0 = address of string in memory	
read_int	5		integer returned in \$vo
read_float	6		float returned in \$vo
read_double	7		double returned in \$vo
read_string	8	\$a0 = memory address of string input buffer \$a1 = length of string buffer (n)	

Figure1: The System calls parameters and their values